

GeoTiff Processor

What the app is, in one sentence

It's a **Flask web app** that lets you **upload GeoTIFFs**, generates **PNG previews** so Leaflet can display them on a web map, lets you **toggle/opacity/zoom** per layer, and can **merge multiple GeoTIFFs into one output GeoTIFF**.

Python (Flask + Rasterio) side

1) Imports and "optional dependency" detection

```
try:
    import rasterio
    ...
    RASTERIO_AVAILABLE = True
except ImportError:
    RASTERIO_AVAILABLE = False
```

- Your app can technically run without Rasterio, but:
 - **previews won't work**
 - **merge won't work**
- Rasterio is the core geospatial raster library here.

Similarly, GDAL is checked but not actually used directly in the logic (Rasterio itself is built on GDAL).

2) Flask app setup + absolute folder paths

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
app.config['UPLOAD_FOLDER'] = os.path.join(BASE_DIR, 'uploads')
app.config['OUTPUT_FOLDER'] = os.path.join(BASE_DIR, 'outputs')
```

This prevents the classic issue:

- You start the server from a different working directory,
- files are saved in one location,
- and the preview route looks in a different location,
- so Leaflet throws "Failed to load preview."

Using absolute paths makes "save" and "serve" reliably point to the same folders.

3) In-memory layer store

```
LAYER_STORE = {}
```

- When a file is uploaded, the server assigns a unique `layer_id`.
- It stores the layer metadata and file paths in `LAYER_STORE[layer_id]`.
- This avoids guessing filenames later when merging.

Downside: if server restarts, this resets. That's fine for demos, not for "enterprise."

TIFFProcessor class (core logic)

A) `get_tiff_info(filepath)` (extract metadata + bounds)

Purpose:

- Read width/height, band count, CRS, and geographic bounds.

Important bits:

Width/height/bands/crs

```
info['width'] = src.width  
info['height'] = src.height  
info['bands'] = src.count  
info['crs'] = src.crs.to_string()
```

Bounds + WGS84 conversion

Leaflet expects geographic lat/long in **EPSG:4326**.

GeoTIFFs are often projected (like UTM), so bounds must be converted:

```
b = transform_bounds(src.crs, "EPSG:4326", ...)
info['bounds'] = [west, south, east, north]
```

That output is exactly what Leaflet needs to place an image overlay.

If CRS is missing:

- bounds cannot be trusted
- you refuse to display it

B) `create_preview(filepath, output_path, max_size=2048)`

This is the “make it visible in Leaflet” function.

Leaflet cannot draw GeoTIFF directly. It draws PNG/JPEG images.

So you generate a preview PNG.

Step 1: Downsample for performance

```
scale = min(max_size / src.width, max_size / src.height, 1.0)
data = src.read(out_shape=(src.count, out_h, out_w), resampling=Resampling.bilinear)
```

- Reads the raster at a smaller resolution.
- Prevents browser meltdown.

Step 2: Convert bands into RGB

If TIFF has ≥ 3 bands: use first 3.

If 1 band: replicate it into 3 channels.

Step 3: Transparent background

This is the key you asked for.

GeoTIFFs often have NoData areas (black/white margins). Those should become transparent.

Best approach:

```
mask = src.dataset_mask(...)
alpha = mask.astype(np.uint8)
```

Rasterio's `dataset_mask()` returns 0–255 mask derived from:

- nodata
- internal masks
- alpha band if present

So NoData becomes alpha=0 (transparent), valid data alpha=255.

Fallbacks if mask fails:

- use nodata value
- or treat zeros as empty

Step 4: Normalize RGB values to 0–255

Raster bands can be huge range (0–10,000 etc). Browsers need 0–255.

So you do min/max normalization per band:

```
band = (band - min) / (max - min)
rgb = rgb * 255
```

Step 5: Save PNG

```
img.save(output_path, "PNG")
```

This is what Leaflet loads.

C) `merge_tiffs(input_files, output_file)`

Purpose:

- Create one GeoTIFF from multiple rasters.

Step 1: Use first file as reference CRS

```
ref_crs = ref.crs
```

Step 2: Ensure every raster is in same CRS

If another file has a different CRS:

```
vrt = WarpedVRT(src, crs=ref_crs)
```

WarpedVRT is "virtual reprojection" on-the-fly.

Step 3: Merge

```
mosaic, out_transform = merge(datasets)
```

Step 4: Write output GeoTIFF

```
with rasterio.open(output_file, "w", **out_meta) as dest:  
    dest.write(mosaic)
```

You set compression LZW to reduce size.

Flask routes

/ (index)

Returns the HTML template with Leaflet UI.

/upload (POST)

When user uploads a TIFF:

1. Create a unique layer id:

```
layer_id = f"layer_{timestamp}_{pid}"
```

2. Save uploaded file into `uploads/`:

```
stored_name = f"{layer_id}_{original_filename}"
```

3. Read metadata + bounds via `get_tiff_info()`

4. Generate preview PNG in `outputs/` :

```
preview_filename = f"{layer_id}_preview.png"
```

5. Save metadata into `LAYER_STORE[layer_id]`
6. Return JSON to browser:

```
{
  "success": true,
  "layer": {
    "id": "...",
    "bounds": [...],
    "preview_url": "/preview/....png"
  }
}
```

That `preview_url` is what Leaflet overlays.

`/preview/<filename>`

Serves the PNG preview file so Leaflet can load it.

Uses `send_file_compat()` because Flask 1.x and 2.x have different parameter names.

`/merge` (POST)

Takes list of `layer_ids` .

1. Finds file paths using `LAYER_STORE` (no guessing filenames)
2. Output path logic:
 - If user enters a path, use it
 - otherwise default to `outputs/merged_output.tif`
3. For safety, it restricts save location to inside project folder (`BASE_DIR`).
This prevents “write anywhere on server” security issues.
4. Calls `merge_tiffs()`
5. Returns JSON with:

- `saved_to` = absolute server-side path
- `download_url` only if output saved inside outputs folder

/download/<filename>

Only downloads files from `outputs/`.

Browsers treat it like a real download because:

```
as_attachment=True
```

JavaScript (Leaflet UI) side

1) Create map

```
const map = L.map('map', { zoomControl: false }).setView([20, 0], 2);
```

- `zoomControl: false` removes the built-in left-side zoom buttons.
- You keep your custom buttons on the right.

2) Base map switching

You define two tile layers:

- Satellite (Esri)
- Street (OpenStreetMap)

Switching:

```
map.removeLayer(currentBaseLayer);  
currentBaseLayer = ...  
currentBaseLayer.addTo(map);
```

3) Upload logic

User selects file(s).

For each file:

- Put into `FormData`
 - POST to `/upload`
 - Receive JSON response layer metadata
 - Add to local `layers[]`
 - Call `addLayerToMap(layer)`
-

4) Display TIFF overlay as image overlay

Leaflet cannot load TIFF directly. So you use preview PNG:

```
const overlay = L.imageOverlay(layer.preview_url, bounds, { opacity: 0.7 })
overlay.addTo(map)
```

`bounds` comes from Python as WGS84 bounds.

5) Auto-zoom to uploaded raster

```
map.flyToBounds(bounds, { padding: [50,50], duration: 1.5 })
```

This “flies” the map to the raster.

6) Layer UI controls

For each layer you render:

- zoom to layer
- toggle visibility
- remove
- opacity slider

Opacity slider:

```
overlay.setOpacity(value / 100)
```

Toggle:


```
map.removeLayer(overlay) / overlay.addTo(map)
```

7) Merge button

When clicked:

1. Collect all layer IDs:

```
layers.map(l => l.id)
```

1. Read output path from input:

```
document.getElementById('mergeOutputPath').value
```

1. POST JSON to `/merge`
2. If success:
 - show alert with `saved_to`
 - if there's a download URL:

```
window.location.href = data.download_url;
```

This triggers a Chrome-style download (not new tab).

Why previews are needed at all

Boss-friendly explanation:

- GeoTIFF is a geospatial raster format, not natively displayable by browsers.
- Leaflet displays images and tiles.
- So we render a downsampled PNG preview and overlay it with geographic bounds.
- That's the bridge between "GIS world" and "web map world."

Limitations you should mention (so you look honest and competent)

1. Layer store is in-memory

- server restart loses layers
- production version would store metadata in a DB or persist a manifest JSON file

2. Preview normalization is simple

- For scientific imagery, you might want histogram stretching or gamma correction.

3. Merge output download only works if output is inside outputs/

- deliberate safety decision

4. Very large rasters

- Downsampled preview works
- But for true performance, you'd generate map tiles (XYZ tiles) instead of image overlays